

RDL TECHNOLOGIES LTD.

Adyar, Mangalore

PROJECT DOCUMENTATION*Internship Technical Report***Embedded Web Server Using STM32F446RCTx & W5500 Ethernet**

Task 3 — Converting HTML to C Byte Array Using Perl

Department:	Embedded Systems / IoT
Microcontroller:	STM32F446RCTx (LQFP64)
Ethernet Module:	WIZnet W5500 (SPI Interface)
IDE Used:	STM32CubeIDE + STM32CubeMX
Task:	Task 3 — HTML to C Byte Array Conversion via Perl Script
Year:	2026

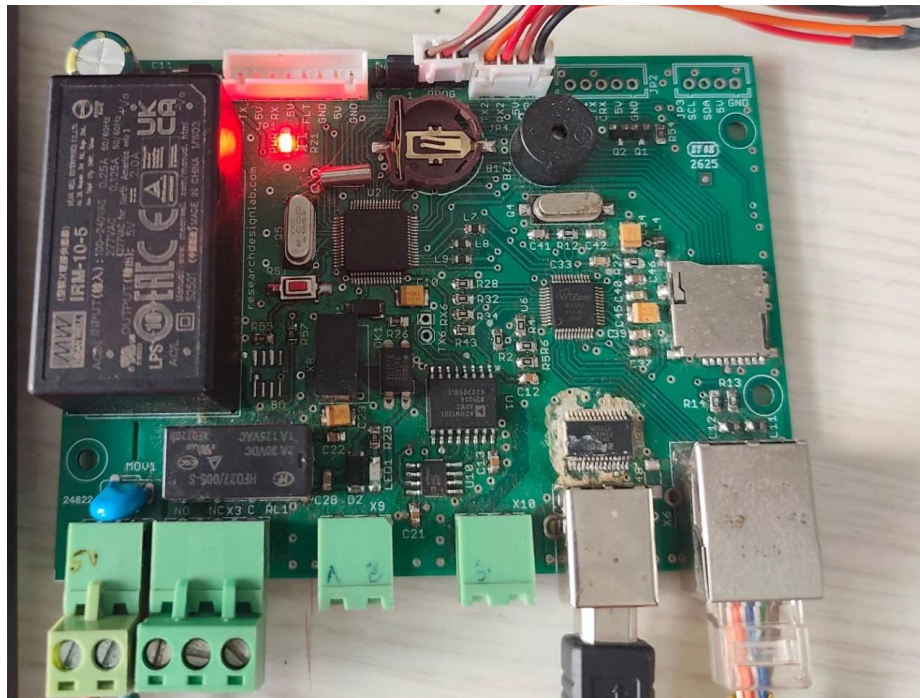


Figure 1: STM32F446RCTx custom board powered via 100-240V AC mains supply through onboard IRM-10-5 AC-DC converter module

Task 3 — Converting HTML to C Byte Array Using Perl

In Task 3 (Network Ping Test), the board successfully responded to ping commands from the PC. Task 3 builds directly on that foundation. The objective is to serve a complete HTML web page from the STM32 board so that any browser on the same network can view the RDL Technologies portal by simply typing the board's IP address.

The STM32 cannot store or read HTML files the way a regular web server does. Instead, the entire HTML file must be converted into a C byte array and compiled directly into the firmware. This section explains how to do that conversion using a Perl script.

What Task 3 Achieves

A Perl script reads the index.html file byte-by-byte and produces two C source files:

- `webpage.c` — contains the full HTML content as a C unsigned char array
- `webpage.h` — declares the array and its length for use in `main.c`

These files are added to the STM32CubeIDE project. When a browser connects to the board, `main.c` sends the byte array directly over TCP — the browser receives it and renders the HTML page.

1 Why a Byte Array?

The STM32 has no file system and no SD card in this project. The only memory available for storing data is Flash memory, which is part of the chip itself. Flash memory is read-only at runtime — you cannot write new files into it while the program is running.

The solution is to encode the HTML file as a C array of bytes (unsigned char values) at compile time. The compiler places this array in Flash. At runtime, `main.c` reads the array and sends it over the Ethernet socket. The browser on the other end receives the bytes and renders them as a normal HTML page.

Normal Web Server	STM32 Embedded Server
Reads index.html from disk at runtime	HTML is compiled into firmware as a byte array
Can update the file without recompiling	Changing the HTML requires re-running the Perl script and recompiling
Requires an OS and file system	Works directly on bare-metal STM32 with no OS

2 What is Perl?

Perl is a general-purpose scripting language that is especially good at reading files, processing text, and automating repetitive tasks. It is free and open-source. In this project, Perl is used to read the HTML file one byte at a time and print each byte as a hex value into a C source file.

2.1 Installing Strawberry Perl on Windows

Strawberry Perl is the recommended Perl distribution for Windows. It includes everything needed to run Perl scripts — the Perl interpreter, standard libraries, and command-line tools. Follow the steps below to install it.

Download Link

Go to <https://strawberryperl.com> — Download the latest 64-bit MSI installer (e.g. strawberry-perl-5.38.x.x-64bit.msi). The file is approximately 80 MB.



*"When I'm on Windows, I use Strawberry Perl!"
-- Larry Wall*

The Perl for MS Windows, free of charge!

Perl is a programming language suitable for writing simple scripts as well as complex applications — see <https://www.perl.org>.

Strawberry Perl is a perl environment for MS Windows containing all you need to run and develop perl applications. It is designed to be as close as possible to perl environment on UNIX systems.

It includes perl binaries, compiler (gcc) + related tools, all the external libraries (crypto, math, graphics, xml...), all the bundled database clients and all you expect from Strawberry Perl.

Latest Release: 5.42.2.1 (2026-04-07)

5.42.2.1 MSI (200.1 MB)
Sha256: 1d473a709e4669128f52b30426a423144a1b1c0f4460e8804f519f534c6583
5.42.2.1 Portable zip (270.2 MB)
Sha256: 32d83be90cf04b07cf9477482bc36302cde6f5b04c574b1adebcb8f07898
5.42.2.1 PDL zip (310.8 MB)
Sha256: 5969e7dbd20ef6d1bc0e5db2b182158686c596effa45a65ac5d2e39c6d6e2

Installation Steps:

1. Run the downloaded .msi installer file.
2. Click Next on the welcome screen.
3. Accept the license agreement and click Next.
4. Leave the installation folder as the default (C:\Strawberry) and click Next.
5. Click Install. The installation will take 2 to 3 minutes.
6. Click Finish when the installer completes.
7. Open a new Command Prompt (cmd) and type the command below to verify the installation was successful:

```
perl --version
```

You should see output similar to the following. The exact version number may differ, but the format will be the same:

```
This is perl 5, version 38, subversion 2 (v5.38.2)
built for MSWin32-x64-multi-thread

Copyright 1987-2023, Larry Wall
```

Tip — If 'perl' is not recognised

Close the Command Prompt window completely and open a new one. Strawberry Perl adds itself to the system PATH during installation, but the change only takes effect in newly opened windows.

If the problem persists, restart your PC and try again.

— Successful Perl Version Check

```
C:\WINDOWS\system32\cmd. x + v
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\YASHASVI>perl --version

This is perl 5, version 40, subversion 2 (v5.40.2) built for MSWin32-x64-multi-thread

Copyright 1987-2025, Larry Wall

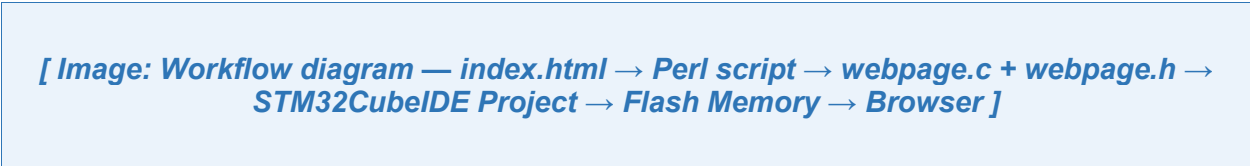
Perl may be copied only under the terms of either the Artistic License or the
GNU General Public License, which may be found in the Perl 5 source kit.

Complete documentation for Perl, including FAQ lists, should be found on
this system using "man perl" or "perldoc perl".  If you have access to the
Internet, point your browser at https://www.perl.org/, the Perl Home Page.

C:\Users\YASHASVI>
```

3 The Conversion Workflow

The complete conversion process takes the index.html file as input and produces two C files that are then added to the STM32CubeIDE project. The diagram below summarises the flow:



Step	Action	Result
1	Prepare index.html	Your complete HTML web page file is ready in a working folder.
2	Place Perl script in the same folder	Both html2c.pl and index.html must be in the same directory.
3	Run the Perl script	webpage.c and webpage.h are generated in the same folder.
4	Add files to STM32CubeIDE	Copy webpage.c to Core/Src and webpage.h to Core/Inc in the project.
5	Build and flash	Compile and upload the firmware. The HTML is now in Flash memory.

4 The Perl Script — html2c.pl

Save the following script as html2c.pl in the same folder as your index.html file. The script reads index.html byte by byte and writes the hex values into webpage.c, then writes the matching header declarations into webpage.h.

```
use strict;
use warnings;

my $input      = "index.html";
my $output_c   = "webpage.c";
my $output_h   = "webpage.h";

open(my $in,    "<", $input)      or die "Cannot open input: $!";
open(my $out_c, ">", $output_c) or die "Cannot open output C: $!";
open(my $out_h, ">", $output_h) or die "Cannot open output H: $!";

# — webpage.c —————
print $out_c "#include \"webpage.h\"\n\n";
print $out_c "const unsigned char index_html[] = {\n    ";

my $count = 0;
while (read($in, my $byte, 1)) {
    printf $out_c "0x%02X,", ord($byte);
    $count++;
    if ($count % 16 == 0) {
        print $out_c "\n    ";
    }
}

print $out_c "\n};\n\n";
print $out_c "const unsigned int index_html_len = $count;\n";

# — webpage.h —————
print $out_h "#ifndef WEBPAGE_H\n";
print $out_h "#define WEBPAGE_H\n\n";
print $out_h "#include <stdint.h>\n\n";
print $out_h "extern const unsigned char index_html[];\n";
print $out_h "extern const unsigned int  index_html_len;\n\n";
print $out_h "#endif /* WEBPAGE_H */\n";

close($in);
close($out_c);
close($out_h);

print "Conversion done!  ($count bytes converted)\n";
```

4.1 What Each Section of the Script Does

Script Section	What It Does
use strict; use warnings;	Enables strict error checking so typos and mistakes cause clear error messages instead of silent failures.
open(\$in, "<", \$input)	Opens index.html for reading. The script will die with a clear message if the file is not found.

<code>open(\$out_c, ">", \$output_c)</code>	Creates (or overwrites) webpage.c for writing. Same for webpage.h.
<code>read(\$in, my \$byte, 1)</code>	Reads exactly one byte from the HTML file at a time. Returns 0 at end of file, which stops the loop.
<code>printf \$out_c "0x%02X,"</code>	Converts the byte to its hex representation. For example, the letter 'A' (ASCII 65) becomes 0x41.
<code>\$count % 16 == 0</code>	After every 16 hex values, a new line is inserted to keep the generated C file readable.
<code>index_html_len = \$count</code>	Writes the total byte count at the end of webpage.c. main.c uses this value to know how many bytes to send over the socket.

5 Running the Script

Place html2c.pl and index.html in the same folder on your PC. Open a Command Prompt, navigate to that folder, and run the command below.

5.1 Navigate to the Folder

```
cd C:\Users\YourName\Documents\WebserverProject
```

Replace the path above with the actual path to your folder.

5.2 Confirm Both Files Are Present

```
dir
```

You should see both index.html and html2c.pl listed in the output.

5.3 Run the Perl Script

```
perl html2c.pl
```

If the conversion is successful, you will see this output:

```
Conversion done! (35916 bytes converted)
```

The exact byte count will match the size of your index.html file. A larger HTML file produces a larger byte count.

Important — File Size Limit

The STM32F446RCTx has 256 KB of Flash memory. The firmware code itself occupies part of this, leaving approximately 180–200 KB for the HTML byte array. If your index.html is larger than this, the linker will report an error during build. Keep the HTML file compact — minify it if necessary, by removing unnecessary whitespace and comments before running the Perl script.

6 The Generated Output Files

After running the script, two new files appear in the same folder. The following sections show what each file looks like.

6.1 webpage.c — The Byte Array

This file contains the full HTML content encoded as hexadecimal byte values. Each value is one character or byte from the original HTML file. Below is a shortened example showing just the first few lines:

```
#include "webpage.h"

const unsigned char index_html[] = {

0x3C,0x21,0x44,0x4F,0x43,0x54,0x59,0x50,0x45,0x20,0x68,0x74,0x6D,0x6C,0x3E,0x0A,

0x3C,0x68,0x74,0x6D,0x6C,0x20,0x6C,0x61,0x6E,0x67,0x3D,0x22,0x65,0x6E,0x22,0x3E,

0x3C,0x68,0x65,0x61,0x64,0x3E,0x3C,0x6D,0x65,0x74,0x61,0x20,0x63,0x68,0x61,0x72,
/* ... thousands more bytes ... */
};

const unsigned int index_html_len = 35916;
```

Reading the Hex Values

0x3C is the hex code for '<', 0x21 is '!', 0x44 is 'D', 0x4F is 'O', 0x43 is 'C'.
So the first bytes spell out '<!DOC' — the start of '<!DOCTYPE html>'.
Every character in the HTML file maps to exactly one hex byte in this array.

6.2 webpage.h — The Header File

This file declares the array and its length so that main.c can use them. It uses extern because the actual data is defined in webpage.c, not here.

```
#ifndef WEBPAGE_H
#define WEBPAGE_H

#include <stdint.h>

extern const unsigned char index_html[];
extern const unsigned int index_html_len;

#endif /* WEBPAGE_H */
```

7 Adding the Files to STM32CubeIDE

Once webpage.c and webpage.h are generated, they must be added to the STM32CubeIDE project so that the compiler can include them when building the firmware.

Step-by-Step:

1. In STM32CubeIDE, open your project in the Project Explorer panel on the left side.
2. Copy webpage.c into the Core/Src folder inside your project directory using Windows Explorer.
3. Copy webpage.h into the Core/Inc folder inside your project directory.
4. Back in STM32CubeIDE, press F5 to refresh the Project Explorer. Both files should now appear in the tree.

Note — Prescaler Changed from 4 to 32

In the previous task, the SPI2 prescaler was set to 4, giving a baud rate of 11 MHz. During testing, this high clock speed caused unreliable communication with the W5500 chip. The prescaler was therefore changed to 32, reducing the SPI clock to 1.375 MHz (44 MHz ÷ 32). This slower speed gives the W5500 more stable signal timing across the PCB traces and eliminated the communication errors seen at the higher speed. The W5500 responded correctly and the web server worked reliably after this change.

Prescaler (for Baud Rate)	32 → 1.375 MBits/s	PCLK1 = 44 MHz divided by 32 = 1.375 MHz. Conservative speed for reliable first-time testing. W5500 supports up to 80 MHz.
---------------------------	--------------------	--

9 How main.c Sends the Web Page

When a browser connects to the board on port 80, the socket state machine in main.c enters the SOCK_ESTABLISHED case. The code reads the incoming HTTP request, sends the HTTP response header, and then transmits the byte array in 1 KB chunks.

The relevant section of main.c is shown below:

```
case SOCK_ESTABLISHED:

    if (getSn_RX_RSR(socket_num) > 0)
    {
        memset(buffer, 0, sizeof(buffer));

        int len = recv(socket_num, buffer, sizeof(buffer)-1);

        if (len > 0 && len < sizeof(buffer))
            buffer[len] = '\0';

        printf("%s\r\n", buffer);

        if (strstr((char*)buffer, "GET /data"))
        {
            /* === Dummy values === */
            uint32_t t = HAL_GetTick();

            float temperature = 25.0f + (float)(t % 100) * 0.1f;
            float vibration = 0.10f + (float)(t % 50) * 0.01f;
            uint16_t rpm = 1400 + (t % 200);
            char* status = (t % 9000 < 3000) ? "FAULT" :
                           (t % 9000 < 6000) ? "WARN" : "NORMAL";

            char json[128];
            snprintf(json, sizeof(json),
                "{\n  \"temp\":%.1f,\n  \"vib\":%.2f,\n  \"rpm\":%u,\n  \"status\": \"%s\"\n}",
                temperature, vibration, rpm, status);

            char hdr[128];
            snprintf(hdr, sizeof(hdr),
```

```

        "HTTP/1.1 200 OK\r\n"
        "Content-Type: application/json\r\n"
        "Access-Control-Allow-Origin: *\r\n"
        "Connection: close\r\n"
        "Content-Length: %d\r\n"
        "\r\n",
        (int)strlen(json));

    send(socket_num, (uint8_t*)hdr, strlen(hdr));
    send(socket_num, (uint8_t*)json, strlen(json));
    disconnect(socket_num);
    break; /* ← don't fall through to HTML */
}
if(strstr((char*)buffer,"PB3ON"))
HAL_GPIO_WritePin(GPIOB,GPIO_PIN_3,GPIO_PIN_SET);

if(strstr((char*)buffer,"PB3OFF"))
HAL_GPIO_WritePin(GPIOB,GPIO_PIN_3,GPIO_PIN_RESET);

if(strstr((char*)buffer,"PB4ON"))
HAL_GPIO_WritePin(GPIOB,GPIO_PIN_4,GPIO_PIN_SET);

if(strstr((char*)buffer,"PB4OFF"))
HAL_GPIO_WritePin(GPIOB,GPIO_PIN_4,GPIO_PIN_RESET);

char http_header[] =
"HTTP/1.1 200 OK\r\n"
"Content-Type: text/html\r\n"
"Connection: close\r\n"
"\r\n";

/* Send header */
send(socket_num, (uint8_t*)http_header, strlen(http_header));

/* Send webpage */
uint16_t sent = 0;

while(sent < index_html_len)
{
    uint16_t chunk = (index_html_len - sent > 1024) ? 1024 :
(index_html_len - sent);

    send(socket_num, (uint8_t*)&index_html[sent], chunk);

    sent += chunk;
}
disconnect(socket_num);
}

break;

```

Code Element	Explanation
<code>recv(socket_num, buffer, ...)</code>	Reads the browser's HTTP GET request into the buffer. This must happen before checking the request or sending any response.
<code>http_header[]</code>	The HTTP/1.1 response header tells the browser the content type is HTML and the connection will close after sending.
<code>index_html_len</code>	The total number of bytes in the HTML array, written by the Perl script into webpage.c at conversion time.
<code>chunk = 1024</code>	The W5500 has limited internal TX buffer memory (2 KB per socket in this config). Sending in 1 KB chunks avoids overflow.
<code>disconnect(socket_num)</code>	Closes the TCP connection after the page is sent. The socket state machine will reopen it for the next request.

10 Testing the Web Server

After building and flashing the firmware, follow the steps below to verify the web page is being served correctly.

Step-by-Step:

1. Connect the STM32 board to the router or switch using the Ethernet cable.
2. Power on the board and open the serial monitor (PuTTY or Tera Term) at 115200 baud.
3. Confirm the following messages appear in the serial monitor:

```
UART3 Working
Try 0: W5500 Version = 0x04
Network Configured
IP: 192.168.1.112
GW: 192.168.1.1
Ethernet Cable Connected
```

4. On the PC, open Command Prompt and run the ping command to confirm the board is reachable on the network:

```

Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\YASHASVI>ping 192.168.1.112

Pinging 192.168.1.112 with 32 bytes of data:
Reply from 192.168.1.112: bytes=32 time=3ms TTL=127
Reply from 192.168.1.112: bytes=32 time=12ms TTL=127
Reply from 192.168.1.112: bytes=32 time=4ms TTL=127
Reply from 192.168.1.112: bytes=32 time=2ms TTL=127

Ping statistics for 192.168.1.112:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
    Approximate round trip times in milli-seconds:
        Minimum = 2ms, Maximum = 12ms, Average = 5ms

C:\Users\YASHASVI>

```

5. On a PC on the same network, open a web browser and navigate to:

```
http://192.168.1.112
```

6. The STM32 board receives the browser's HTTP GET request over the Ethernet connection. The serial monitor will display the raw HTTP headers sent by the browser, confirming that the board is receiving and processing the request correctly:

```

GET /data HTTP/1.1
Host: 192.168.1.112
Connection: keep-alive
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36
Accept: */*
Referer: http://192.168.1.112/
Accept-Encoding: gzip, deflate
Accept-Language: en-IN,en-GB;q=0.9,en-US;q=0.8,en;q=0.7,hi;q=0.6,kn;q=0.5

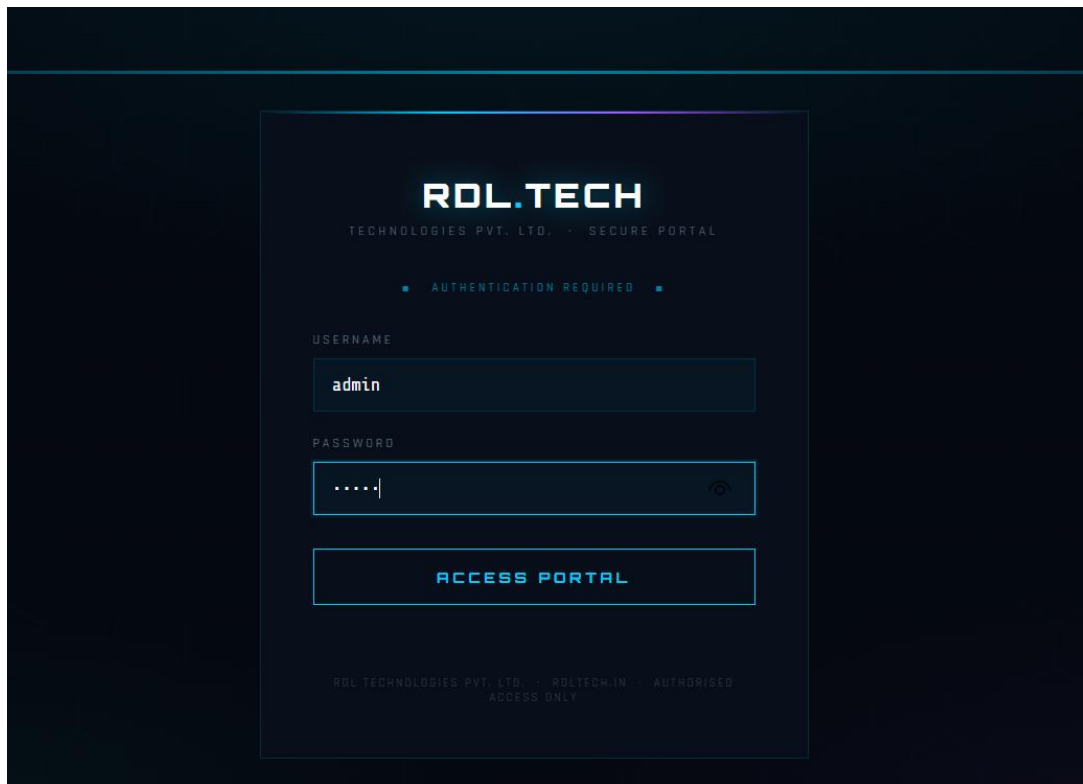
GET /data HTTP/1.1
Host: 192.168.1.112
Connection: keep-alive
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36
Accept: */*
Referer: http://192.168.1.112/
Accept-Encoding: gzip, deflate
Accept-Language: en-IN,en-GB;q=0.9,en-US;q=0.8,en;q=0.7,hi;q=0.6,kn;q=0.5

GET /data HTTP/1.1
Host: 192.168.1.112
Connection: keep-alive
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36
Accept: */*
Referer: http://192.168.1.112/
Accept-Encoding: gzip, deflate
Accept-Language: en-IN,en-GB;q=0.9,en-US;q=0.8,en;q=0.7,hi;q=0.6,kn;q=0.5

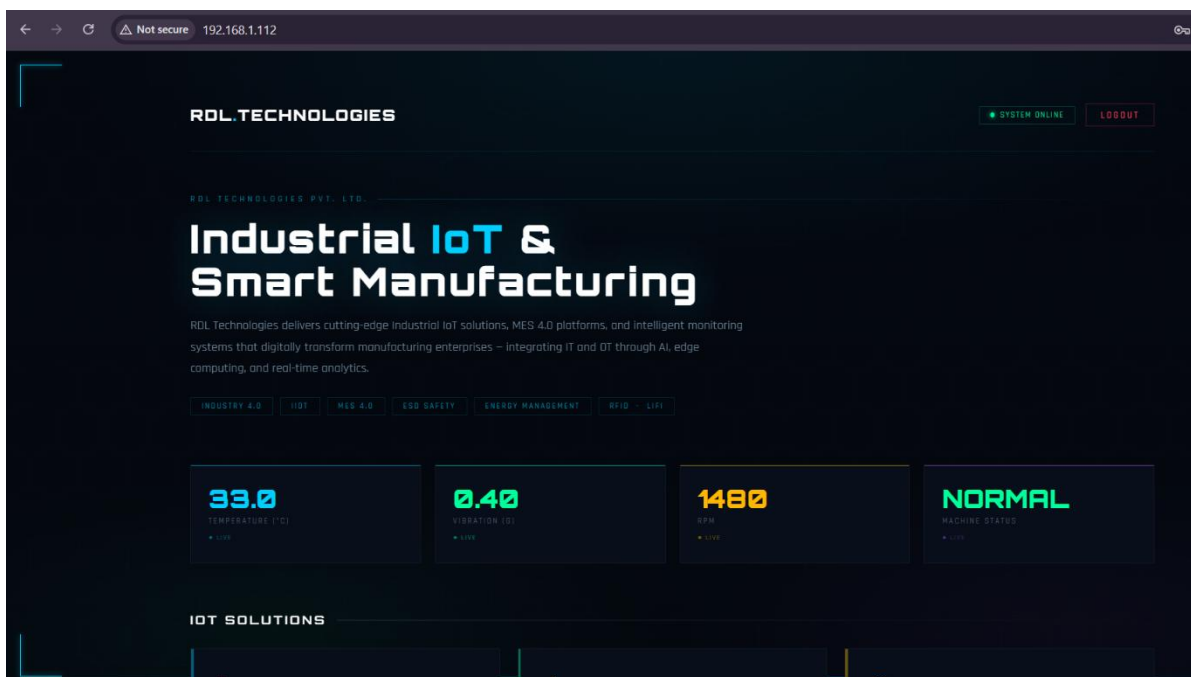
GET /data HTTP/1.1
Host: 192.168.1.112
Connection: keep-alive
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36
Accept: */*
Referer: http://192.168.1.112/
Accept-Encoding: gzip, deflate
Accept-Language: en-IN,en-GB;q=0.9,en-US;q=0.8,en;q=0.7,hi;q=0.6,kn;q=0.5

```

7. The RDL Technologies portal should appear in the browser window.
8. Log in with username: admin and password: admin to access the company dashboard.



Browser showing the RDL Technologies portal login page served from the STM32 board



Browser showing the RDL Technologies company dashboard after successful login]

What If the Page Does Not Load?

1. Check the serial monitor — confirm 'Ethernet Cable Connected' is shown.
2. Run: `ping 192.168.1.112` from the PC Command Prompt. If pings fail, the network is not set up correctly.
3. Confirm the PC and board are on the same subnet (both 192.168.1.x with subnet 255.255.255.0).
4. Check that the gateway address in `main.c` matches your router's IP.
5. Try a different browser or disable browser cache (Ctrl+Shift+R to force reload).

11 Summary

Task 3 completes the embedded web server project. The workflow from HTML file to live web page involves three stages: writing the HTML, converting it with the Perl script, and flashing it into the STM32 board. The table below summarises the complete picture:

Stage	What You Do	What the Board Does
HTML Design	Write and save <code>index.html</code>	Nothing yet — board is not involved
Perl Conversion	Run: <code>perl html_to_c.pl</code>	<code>webpage.c</code> and <code>webpage.h</code> are generated
Build & Flash	Compile project and flash firmware	HTML byte array is stored in Flash memory
Live Serving	Open browser → type board's IP address	Board serves the HTML page over TCP port 80